

AVR TIMER0 & TIMER2 USER MANUAL

By Andrew Ghobrial (aka Heretic)
Documentation v1.3 • Feb 2026

I. Overview

This manual provides a step-by-step guide for using the 8-bit Timer drivers (Timer0 and Timer2). These drivers allow for precise timing, frequency generation (CTC), and Pulse Width Modulation (PWM) on AVR microcontrollers like the ATmega32.

Pin Mapping:

- Timer0 Output (OC0): **PB3**
- Timer2 Output (OC2): **PD7**

II. The "Step-by-Step" Setup

Step 1 Initialization

Before doing anything, you must reset the timer registers and disable interrupts to start from a clean state.

```
T0_init(); // For Timer0  
T2_init(); // For Timer2
```

Step 2 Select Operation Mode

Decide how you want the timer to behave. Use the predefined macros in the header file.

- `T0_MODE_NORMAL` : Simple counter, overflows at 255.
- `T0_MODE_CTC` : Resets when it reaches a specific value (Clear Timer on Compare).
- `T0_MODE_FASTPWM` : High-speed PWM for motor/LED control.

```
T0_setMode(T0_MODE_CTC);
```

Step 3 Set Output Action

If you are using the hardware pins (OC0/OC2), define what happens when the timer hits the target value.

```
T0_setOutputAction(T0_OUT_TOGGLE); // Flips the pin state on every match
```

Step 4 Set Compare Value

Set the "target" number (0-255). In CTC mode, this determines the frequency. In PWM, this determines the duty cycle.

```
T0_setCompareValue(156); // Hits target at 156
```

Step 5 Interrupt Management

If you want the CPU to stop and run a specific function (ISR) when the timer hits the target or overflows, enable interrupts.

```
T0_manageInterrupts();
```

Don't forget to call `sei();` in your main code to enable global interrupts!

Step 6 Start the Timer

The timer only begins counting once you apply a clock source (Prescaler).

```
T0_start(T0_PRE_64); // Timer is now running!
```

III. Quality of Life: Quick PWM

If you just need a standard PWM signal without manually going through all steps, use the wrapper function:

```
T0_doPWM(50, T0_PRE_64); // 50% Duty Cycle, 64 Prescaler
```

Calculations

PWM Frequency is calculated as follows:

$$f_{\text{PWM}} = F_{\text{CPU}} / (\text{Prescaler} * 256)$$

IV. Prescaler Reference Table

Macro	Clock Divisor	Result (at 16MHz)
T0_PRE_1	None	16.0 MHz
T0_PRE_16/64	64	250 kHz
T0_PRE_256	256	62.5 kHz
T0_PRE_1024	1024	15.6 kHz

V. ISR Implementation (In main.c)

To use interrupts, add these handlers to your main file:

```
// Run this when TCNT hits OCR
ISR(TIMER0_COMP_vect) {
    // Your code here (e.g., Toggle a different LED)
}
```

```
// Run this when timer rolls over from 255 to 0
ISR(TIMER0_OVF_vect) {
    // Your code here
}
```